Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means SoA is faster; **bold** marks >1.1×.

Program	**ADT**	**SoA**	**Fold passes**			**Map passes**		
	fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
OctTree	16	9	0.4604	0.4856	0.95×	0.3239	0.3745	0.86×
Compiler	9	8	0.1519	0.0676	**2.25**×	0.1616	0.1792	0.90×
DBQuery	15	13	0.1336	0.0961	**1.39**×	0.1382	0.1693	0.82×
DecisionTree	7	6	3.898	2.985	**1.31**×	–	–	–
DomTree	15	14	0.1755	0.0630	**2.79**×	0.0602	0.0814	0.74×
KDTree	17	16	0.1860	0.1429	**1.30**×	–	–	–
LinearListReduction	11	11	0.0329	0.0057	**5.73**×	–	–	–
List	2	2	0.1299	0.1134	**1.15**×	0.2567	0.4590	0.56×
MonoTree	3	2	0.0173	0.0223	0.78×	0.0839	0.0766	1.10×
ObjectGraph	5	4	0.2190	0.0910	**2.41**×	0.1440	0.1619	0.89×
PiecewiseFunctions	8	7	0.1456	0.0959	**1.52**×	0.2432	0.2638	0.92×
TernaryTree	5	3	0.0286	0.0270	1.06×	0.0979	0.0911	1.07×
Trie	10	9	0.0764	0.0441	**1.73**×	0.1511	0.1677	0.90×

Table 2: PAPI speedup summary across all passes. For each program and counter, speedup is AoS/SoA using summed per-pass median counters. >1× means SoA has fewer events.

Program	**CYC**	**INS**	**L1D**	**L1I**	**L2D**	**L2I**	**LLC**
OctTree	1.00×	0.75×	**1.43**×	0.45×	**1.91**×	0.55×	**3.69**×
Compiler	**1.56**×	0.52×	**3.85**×	0.85×	**20.41**×	0.30×	**7.14**×
DBQuery	1.05×	0.47×	**1.32**×	0.46×	**8.16**×	0.21×	**5.80**×
DecisionTree	**1.32**×	0.59×	**18.57**×	**1.38**×	0.93×	**1.92**×	**32.42**×
DomTree	**1.36**×	0.57×	**1.94**×	1.07×	**18.84**×	0.19×	**10.20**×
KDTree	**1.30**×	0.77×	**1.71**×	1.02×	**2.20**×	0.98×	**4.63**×
LinearListReduction	**6.72**×	0.90×	**39.69**×	**2.15**×	**60.36**×	**3.01**×	**10.68**×
List	0.82×	0.80×	**2.84**×	0.45×	0.13×	0.68×	**1.48**×
MonoTree	0.97×	0.85×	0.57×	0.66×	0.36×	0.52×	**1.26**×
ObjectGraph	0.88×	0.75×	**4.33**×	0.76×	**1.50**×	0.88×	**2.44**×
PiecewiseFunctions	**1.14**×	0.60×	**3.02**×	0.87×	**2.88**×	0.40×	**4.40**×
TernaryTree	**1.15**×	0.79×	0.43×	0.51×	0.29×	0.88×	**1.42**×
Trie	1.09×	0.56×	**3.61**×	0.85×	**5.20**×	0.15×	**5.69**×
Geomean	**1.28**×	0.67×	**2.78**×	0.79×	**2.57**×	0.57×	**4.59**×

Table 3: PAPI counter totals across all passes (AoS/SoA). Each entry is the sum of per-pass median counter values.

Program	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
OctTree	2.7e+09/**2.7e+09**	**8.2e+09**/1.1e+10	3.8e+07/**2.6e+07**	**3.9e+05**/8.6e+05	1.0e+06/**5.2e+05**	**1.7e+04**/3.0e+04	6.1e+07/**1.7e+07**
Compiler	9.1e+08/**5.8e+08**	**9.9e+08**/1.9e+09	1.4e+07/**3.5e+06**	**2.8e+05**/3.3e+05	1.8e+06/**8.7e+04**	**1.1e+04**/3.7e+04	5.1e+07/**7.1e+06**
DBQuery	9.4e+08/**8.9e+08**	**1.1e+09**/2.2e+09	4.5e+06/**3.4e+06**	**1.7e+05**/3.7e+05	8.4e+05/**1.0e+05**	**8.0e+03**/3.7e+04	2.1e+07/**3.5e+06**
DecisionTree	2.0e+10/**1.5e+10**	**4.0e+10**/6.8e+10	5.4e+08/**2.9e+07**	7.7e+04/**5.6e+04**	**2.0e+06**/2.1e+06	7.7e+04/**4.0e+04**	1.1e+09/**3.3e+07**
DomTree	7.5e+08/**5.5e+08**	**1.2e+09**/2.2e+09	1.3e+07/**6.9e+06**	1.2e+05/**1.2e+05**	1.3e+06/**6.7e+04**	**5.8e+03**/3.0e+04	5.6e+07/**5.5e+06**
KDTree	9.2e+08/**7.0e+08**	**3.1e+09**/4.0e+09	1.6e+07/**9.2e+06**	1.8e+03/**1.8e+03**	2.4e+05/**1.1e+05**	**1.6e+03**/1.6e+03	4.4e+07/**9.5e+06**
LinearListReduction	1.5e+08/**2.3e+07**	**9.0e+07**/1.0e+08	1.9e+06/**4.7e+04**	6.0e+02/**2.8e+02**	4.3e+05/**7.0e+03**	4.8e+02/**1.6e+02**	1.2e+07/**1.2e+06**
List	**1.3e+09**/1.5e+09	**4.5e+09**/5.6e+09	9.4e+06/**3.3e+06**	**2.6e+05**/5.8e+05	**2.6e+04**/2.0e+05	**8.2e+03**/1.2e+04	4.9e+07/**3.3e+07**
MonoTree	**2.5e+08**/2.6e+08	**9.1e+08**/1.1e+09	**3.5e+05**/6.2e+05	**3.2e+04**/4.9e+04	**5.9e+03**/1.7e+04	**1.8e+03**/3.4e+03	2.2e+06/**1.8e+06**
ObjectGraph	**7.7e+08**/8.8e+08	**3.0e+09**/4.0e+09	1.1e+07/**2.6e+06**	**1.7e+05**/2.3e+05	1.1e+05/**7.2e+04**	**1.3e+04**/1.5e+04	2.1e+07/**8.4e+06**
PiecewiseFunctions	1.3e+09/**1.1e+09**	**2.8e+09**/4.7e+09	1.6e+07/**5.2e+06**	**4.1e+05**/4.7e+05	2.9e+05/**1.0e+05**	**2.1e+04**/5.3e+04	4.5e+07/**1.0e+07**
TernaryTree	4.2e+08/**3.6e+08**	**1.2e+09**/1.5e+09	**5.9e+05**/1.4e+06	**6.8e+04**/1.3e+05	**6.0e+03**/2.1e+04	**2.7e+03**/3.1e+03	3.2e+06/**2.3e+06**
Trie	6.1e+08/**5.6e+08**	**1.2e+09**/2.2e+09	1.0e+07/**2.8e+06**	**1.9e+05**/2.2e+05	2.8e+05/**5.4e+04**	**6.5e+03**/4.2e+04	2.6e+07/**4.5e+06**

Table 4: Per-pass performance for `OctTree`, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
barnesHutPotential	F	11/16	31%	0.0489	0.0473	1.03×
clearFlags	M	15/16	6%	**0.1575**	0.1761	0.89×
countActive	F	10/16	38%	0.0408	**0.0341**	**1.20**×
countParticles	F	8/16	50%	**0.0666**	0.1535	0.43×
fmmPotential	F	12/16	25%	0.0989	0.1025	0.96×
scaleEnergy	M	16/16	0%	**0.1664**	0.1984	0.84×
sumEnergy	F	12/16	25%	0.0744	**0.0377**	**1.97**×
sumMass	F	10/16	38%	0.0336	**0.0300**	**1.12**×
paletteEntriesQuantized	F	13/25	48%	0.0479	**0.0379**	**1.27**×
quantizationErrorProxy	F	10/25	60%	0.0493	**0.0426**	**1.16**×
Total				0.7843	0.8601	0.91×
Geomean						1.02×

Table 5: Per-pass PAPI counters for `OctTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
barnesHutPotential	F	2.4e+08/**2.1e+08**	**8.2e+08**/1.0e+09	6.3e+06/**4.2e+06**	**6.0e+02**/6.1e+02	2.0e+05/**8.6e+04**	**4.1e+02**/4.1e+02	5.6e+06/**1.7e+06**
clearFlags	M	**3.7e+08**/4.4e+08	**8.0e+08**/1.7e+09	**1.3e+06**/2.3e+06	**2.2e+05**/4.2e+05	**1.4e+04**/3.3e+04	**6.4e+03**/1.3e+04	5.6e+06/**2.4e+06**
countActive	F	2.0e+08/**1.6e+08**	5.8e+08/**5.8e+08**	3.5e+06/**9.1e+05**	6.7e+02/**4.6e+02**	1.2e+05/**2.3e+04**	5.4e+02/**2.8e+02**	6.0e+06/**3.0e+05**
countParticles	F	1.7e+08/**1.5e+08**	5.7e+08/**5.7e+08**	7.4e+06/**2.3e+05**	8.4e+02/**5.4e+02**	2.9e+05/**8.2e+03**	6.7e+02/**2.2e+02**	4.7e+06/**1.4e+05**
fmmPotential	F	**4.9e+08**/5.1e+08	**1.6e+09**/1.8e+09	**8.9e+05**/7.2e+06	**6.5e+02**/1.1e+03	**1.7e+04**/1.4e+05	**5.4e+02**/8.9e+02	4.4e+06/**2.0e+06**
paletteEntriesQuantized	F	2.4e+08/**1.7e+08**	**6.7e+08**/7.6e+08	6.2e+06/**1.2e+06**	**5.8e+02**/7.2e+02	1.3e+05/**4.1e+04**	**4.1e+02**/4.9e+02	9.3e+06/**1.4e+06**
quantizationErrorProxy	F	2.5e+08/**2.2e+08**	**9.7e+08**/1.2e+09	4.1e+06/**1.8e+06**	**2.1e+02**/4.1e+02	**1.5e+04**/3.3e+04	**1.9e+02**/4.0e+02	9.3e+06/**2.6e+06**
scaleEnergy	M	**3.5e+08**/4.6e+08	**9.1e+08**/1.9e+09	**1.6e+06**/2.8e+06	**1.6e+05**/4.4e+05	**1.2e+04**/4.5e+04	**6.6e+03**/1.4e+04	5.5e+06/**2.5e+06**
sumEnergy	F	**1.8e+08**/1.8e+08	**7.7e+08**/8.9e+08	5.8e+06/**5.1e+06**	5.9e+02/**5.5e+02**	2.0e+05/**1.0e+05**	4.8e+02/**3.6e+02**	4.8e+06/**2.3e+06**
sumMass	F	1.7e+08/**1.4e+08**	**5.7e+08**/6.3e+08	8.0e+05/**6.4e+05**	5.9e+02/**4.0e+02**	**2.6e+03**/1.6e+04	4.4e+02/**2.4e+02**	6.3e+06/**1.2e+06**
Total		2.7e+09/2.7e+09	8.2e+09/1.1e+10	3.8e+07/2.6e+07	3.9e+05/8.6e+05	1.0e+06/5.2e+05	1.7e+04/3.0e+04	6.1e+07/1.7e+07

Table 6: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
blockCountPass	F	2/9	78%	0.0134	**0.0037**	**3.59**×
branchStatsPass	F	2/9	78%	0.0120	**0.0048**	**2.51**×
castInstCountPass	F	3/9	67%	0.0187	**0.0022**	**8.68**×
has cycle	F	4/9	56%	0.0352	0.0334	1.06×
instCountPass	F	2/9	78%	0.0150	**0.0048**	**3.13**×
latencyModelPass	F	3/9	67%	0.0123	**0.0036**	**3.47**×
memoryOpStatsPass	F	3/9	67%	0.0122	**0.0058**	**2.09**×
stripSideEffectsPass	M	7/9	22%	**0.0804**	0.0907	0.89×
targetReturnPass	M	9/9	0%	0.0812	0.0884	0.92×
throughputModelPass	F	3/9	67%	0.0124	**0.0030**	**4.16**×
verifyIR	F	9/9	0%	0.0206	**0.0064**	**3.24**×
Total				0.3135	0.2467	**1.27**×
Geomean						**2.47**×

Table 7: Per-pass PAPI counters for `Compiler` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
blockCountPass	F	5.0e+07/**9.8e+06**	5.1e+07/**4.5e+07**	1.4e+06/**7.2e+03**	2.5e+02/**8.4e+01**	2.6e+05/**1.1e+03**	2.1e+02/**6.8e+01**	4.7e+06/**1.4e+04**
branchStatsPass	F	6.1e+07/**1.6e+07**	**7.9e+07**/8.6e+07	8.2e+05/**2.2e+04**	2.3e+02/**1.6e+02**	5.4e+04/**3.4e+03**	2.1e+02/**1.4e+02**	4.7e+06/**4.8e+05**
castInstCountPass	F	9.5e+07/**6.6e+06**	5.1e+07/**3.9e+07**	**1.3e+04**/3.3e+04	1.3e+02/**9.3e+01**	**8.7e+02**/1.2e+03	1.2e+02/**7.9e+01**	4.7e+06/**1.4e+04**
has cycle	F	1.3e+08/**8.9e+07**	**1.1e+08**/1.5e+08	2.4e+06/**5.6e+05**	8.4e+04/**5.9e+04**	1.2e+05/**2.2e+04**	2.1e+03/**6.2e+02**	5.0e+06/**1.5e+06**
instCountPass	F	4.2e+07/**1.2e+07**	**5.6e+07**/5.7e+07	1.4e+06/**1.4e+04**	2.2e+02/**1.4e+02**	3.2e+05/**1.2e+03**	2.0e+02/**1.2e+02**	4.7e+06/**4.4e+04**
latencyModelPass	F	6.3e+07/**1.3e+07**	**6.2e+07**/6.9e+07	1.2e+06/**3.7e+04**	1.8e+02/**1.1e+02**	2.1e+05/**4.0e+03**	1.6e+02/**9.1e+01**	4.8e+06/**6.1e+05**
memoryOpStatsPass	F	6.0e+07/**1.7e+07**	**8.5e+07**/9.1e+07	6.7e+05/**3.1e+04**	2.4e+02/**1.0e+02**	3.3e+04/**3.3e+03**	2.1e+02/**9.2e+01**	4.7e+06/**4.8e+05**
stripSideEffectsPass	M	**1.5e+08**/2.0e+08	**1.8e+08**/6.0e+08	1.9e+06/**1.3e+06**	**1.0e+05**/1.4e+05	**6.4e+03**/1.9e+04	**3.2e+03**/1.4e+04	4.3e+06/**1.2e+06**
targetReturnPass	M	**1.5e+08**/1.9e+08	**1.7e+08**/6.3e+08	1.6e+06/**1.5e+06**	**9.3e+04**/1.3e+05	**5.4e+03**/2.4e+04	**3.7e+03**/2.2e+04	3.7e+06/**1.5e+06**
throughputModelPass	F	6.3e+07/**1.5e+07**	**6.2e+07**/6.9e+07	1.2e+06/**3.0e+04**	2.5e+02/**1.3e+02**	2.1e+05/**3.6e+03**	2.1e+02/**9.6e+01**	4.8e+06/**6.8e+05**
verifyIR	F	4.2e+07/**1.4e+07**	**7.4e+07**/8.6e+07	1.1e+06/**2.0e+04**	9.1e+02/**6.4e+02**	5.5e+05/**4.1e+03**	8.1e+02/**5.1e+02**	4.7e+06/**4.9e+05**
Total		9.1e+08/5.8e+08	9.9e+08/1.9e+09	1.4e+07/3.5e+06	2.8e+05/3.3e+05	1.8e+06/8.7e+04	1.1e+04/3.7e+04	5.1e+07/7.1e+06

Table 8: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
clearQueryFlags	M	14/15	7%	**0.0681**	0.0860	0.79×
countJoins	F	3/15	80%	0.0193	**0.0116**	**1.66**×
filterSelectivitySkew	F	5/15	67%	0.0194	**0.0122**	**1.59**×
hashJoinPressure	F	5/15	67%	0.0197	**0.0124**	**1.60**×
scaleCosts	M	15/15	0%	**0.0701**	0.0834	0.84×
sumCost	F	6/15	60%	0.0291	**0.0220**	**1.32**×
sumMemory	F	6/15	60%	0.0192	**0.0153**	**1.25**×
sumRows	F	6/15	60%	0.0269	**0.0226**	**1.19**×
Total				0.2718	0.2654	1.02×
Geomean						**1.24**×

Table 9: Per-pass PAPI counters for `DBQuery` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
clearQueryFlags	M	**1.6e+08**/2.2e+08	**1.9e+08**/7.2e+08	**3.1e+05**/1.5e+06	**9.2e+04**/1.7e+05	3.0e+04/**2.8e+04**	**2.7e+03**/1.7e+04	2.3e+06/**9.3e+05**
countJoins	F	9.8e+07/**5.9e+07**	7.4e+07/**7.1e+07**	6.1e+05/**2.5e+04**	4.3e+02/**2.4e+02**	1.4e+05/**2.0e+03**	4.0e+02/**1.7e+02**	2.7e+06/**5.8e+04**
filterSelectivitySkew	F	9.8e+07/**6.1e+07**	**1.0e+08**/1.1e+08	5.1e+05/**6.1e+04**	2.1e+02/**1.5e+02**	1.2e+05/**5.9e+03**	1.9e+02/**1.4e+02**	2.7e+06/**3.3e+05**
hashJoinPressure	F	1.0e+08/**6.3e+07**	**8.4e+07**/9.9e+07	7.2e+05/**9.6e+04**	2.9e+02/**1.5e+02**	1.9e+05/**7.7e+03**	2.6e+02/**1.3e+02**	2.8e+06/**1.4e+05**
scaleCosts	M	**1.6e+08**/2.2e+08	**2.2e+08**/7.5e+08	**3.1e+05**/1.2e+06	**7.6e+04**/2.0e+05	3.5e+04/**2.6e+04**	**3.5e+03**/2.0e+04	2.5e+06/**1.0e+06**
sumCost	F	8.6e+07/**7.5e+07**	**9.0e+07**/1.2e+08	1.1e+06/**3.0e+05**	4.6e+02/**3.1e+02**	1.5e+05/**1.4e+04**	3.5e+02/**2.2e+02**	2.4e+06/**3.5e+05**
sumMemory	F	9.7e+07/**7.8e+07**	**9.0e+07**/1.2e+08	6.6e+05/**7.2e+04**	3.3e+02/**1.6e+02**	1.5e+05/**1.1e+04**	3.0e+02/**1.4e+02**	2.8e+06/**3.8e+05**
sumRows	F	1.3e+08/**1.1e+08**	**2.1e+08**/2.3e+08	2.9e+05/**1.1e+05**	2.5e+02/**2.1e+02**	2.3e+04/**7.5e+03**	2.3e+02/**2.1e+02**	2.4e+06/**3.2e+05**
Total		9.4e+08/8.9e+08	1.1e+09/2.2e+09	4.5e+06/3.4e+06	1.7e+05/3.7e+05	8.4e+05/1.0e+05	8.0e+03/3.7e+04	2.1e+07/3.5e+06

Table 10: Per-pass performance for `DecisionTree`, ADT has 7 fields, SoA uses 6 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
classify Batch	F	5/7	29%	3.345	**2.625**	**1.27**×
classify Depth	F	4/7	43%	0.0478	**0.0368**	**1.30**×
classify tree	F	5/7	29%	0.0479	**0.0371**	**1.29**×
countFeatureUses	F	3/7	57%	0.0510	**0.0319**	**1.60**×
countLeaves	F	2/7	71%	0.0486	**0.0252**	**1.93**×
countNodes	F	2/7	71%	0.0488	**0.0367**	**1.33**×
countSmallLeaves	F	3/7	57%	0.0516	**0.0327**	**1.58**×
inferenceCost	F	2/7	71%	0.0521	**0.0302**	**1.72**×
sumImpurity	F	3/7	57%	0.0510	**0.0334**	**1.53**×
sumPathLengths	F	3/7	57%	0.0537	**0.0346**	**1.55**×
sumSamples	F	3/7	57%	0.0492	**0.0325**	**1.51**×
treeDepth	F	2/7	71%	0.0519	**0.0289**	**1.80**×
Total				3.898	2.985	**1.31**×
Geomean						**1.52**×

Table 11: Per-pass PAPI counters for `DecisionTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
classify Batch	F	1.7e+10/**1.3e+10**	**3.3e+10**/5.9e+10	4.6e+08/**2.6e+07**	6.4e+04/**4.7e+04**	**1.5e+06**/1.9e+06	6.4e+04/**3.2e+04**	9.3e+08/**2.1e+07**
classify Depth	F	2.4e+08/**1.9e+08**	**4.7e+08**/8.5e+08	6.7e+06/**3.6e+05**	9.8e+02/**3.5e+02**	**2.0e+04**/2.8e+04	9.6e+02/**3.0e+02**	1.3e+07/**3.4e+05**
classify tree	F	2.4e+08/**1.8e+08**	**4.7e+08**/8.5e+08	6.6e+06/**3.6e+05**	1.0e+03/**3.5e+02**	**2.4e+04**/2.8e+04	9.8e+02/**3.2e+02**	1.3e+07/**3.5e+05**
countFeatureUses	F	2.6e+08/**1.6e+08**	**7.5e+08**/7.8e+08	7.4e+06/**4.3e+05**	9.2e+02/**6.6e+02**	3.8e+04/**1.4e+04**	8.7e+02/**6.4e+02**	1.5e+07/**2.1e+06**
countLeaves	F	2.5e+08/**1.3e+08**	**6.0e+08**/6.1e+08	7.4e+06/**1.0e+05**	1.0e+03/**6.2e+02**	4.6e+04/**8.0e+03**	9.7e+02/**5.7e+02**	1.4e+07/**4.3e+05**
countNodes	F	2.5e+08/**1.3e+08**	**6.1e+08**/6.2e+08	7.2e+06/**2.8e+05**	1.7e+03/**1.2e+03**	4.9e+04/**1.0e+04**	1.5e+03/**9.6e+02**	1.4e+07/**3.9e+05**
countSmallLeaves	F	2.6e+08/**1.6e+08**	**7.1e+08**/8.4e+08	5.9e+06/**8.4e+04**	8.4e+02/**8.0e+02**	3.0e+04/**1.3e+04**	8.2e+02/**7.9e+02**	1.5e+07/**2.1e+06**
inferenceCost	F	2.6e+08/**1.5e+08**	6.1e+08/**6.1e+08**	8.3e+06/**1.2e+05**	1.2e+03/**1.0e+03**	1.3e+05/**8.0e+03**	1.2e+03/**9.8e+02**	1.4e+07/**4.4e+05**
sumImpurity	F	2.6e+08/**1.6e+08**	**7.6e+08**/7.8e+08	7.6e+06/**4.6e+05**	1.4e+03/**1.2e+03**	3.8e+04/**1.3e+04**	1.4e+03/**1.2e+03**	1.4e+07/**2.1e+06**
sumPathLengths	F	2.7e+08/**1.7e+08**	**8.0e+08**/9.0e+08	7.5e+06/**4.0e+05**	1.6e+03/**7.8e+02**	3.6e+04/**1.4e+04**	1.6e+03/**7.7e+02**	1.5e+07/**2.0e+06**
sumSamples	F	2.5e+08/**1.6e+08**	**6.4e+08**/7.7e+08	6.3e+06/**2.5e+05**	**8.2e+02**/8.8e+02	3.9e+04/**1.4e+04**	**8.0e+02**/8.6e+02	1.5e+07/**2.1e+06**
treeDepth	F	2.6e+08/**1.4e+08**	6.1e+08/**6.1e+08**	8.2e+06/**1.1e+05**	1.3e+03/**1.0e+03**	1.9e+04/**8.1e+03**	1.3e+03/**9.7e+02**	1.4e+07/**4.4e+05**
Total		2.0e+10/1.5e+10	4.0e+10/6.8e+10	5.4e+08/2.9e+07	7.7e+04/5.6e+04	2.0e+06/2.1e+06	7.7e+04/4.0e+04	1.1e+09/3.3e+07

Table 12: Per-pass performance for `DomTree`, ADT has 15 fields, SoA uses 14 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
SumArea	F	6/15	60%	0.0716	**0.0208**	**3.44**×
computeWidths	M	14/15	7%	**0.0306**	0.0449	0.68×
count styled	F	3/15	80%	0.0342	**0.0116**	**2.94**×
find max Bottom	F	5/15	67%	0.0358	**0.0188**	**1.91**×
scaleLayout	M	15/15	0%	**0.0296**	0.0364	0.81×
sumTextWidth	F	3/15	80%	0.0339	**0.0118**	**2.89**×
Total				0.2357	0.1443	**1.63**×
Geomean						**1.77**×

Table 13: Per-pass PAPI counters for `DomTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
SumArea	F	1.1e+08/**9.8e+07**	**2.9e+08**/4.9e+08	4.2e+06/**1.7e+06**	9.6e+02/**5.4e+02**	3.4e+05/**2.9e+04**	8.3e+02/**4.1e+02**	1.2e+07/**1.7e+06**
computeWidths	M	**6.6e+07**/1.4e+08	**1.4e+08**/4.6e+08	**1.3e+06**/1.6e+06	6.6e+04/**6.5e+04**	**8.8e+03**/1.0e+04	**1.1e+03**/1.5e+04	1.3e+06/**4.5e+05**
count styled	F	1.7e+08/**5.9e+07**	**2.6e+08**/2.6e+08	2.0e+06/**2.6e+04**	8.8e+02/**1.2e+02**	3.0e+05/**5.0e+03**	8.6e+02/**1.0e+02**	1.4e+07/**7.2e+05**
find max Bottom	F	1.6e+08/**9.5e+07**	**2.5e+08**/4.1e+08	2.4e+06/**1.3e+06**	4.5e+02/**1.8e+02**	1.7e+05/**9.5e+03**	4.2e+02/**1.4e+02**	1.4e+07/**1.4e+06**
scaleLayout	M	**6.0e+07**/9.8e+07	**8.8e+07**/2.9e+08	**1.1e+06**/2.3e+06	5.5e+04/**5.0e+04**	2.7e+04/**9.2e+03**	**2.0e+03**/1.5e+04	1.4e+06/**4.5e+05**
sumTextWidth	F	1.7e+08/**6.0e+07**	**2.0e+08**/2.4e+08	2.4e+06/**1.8e+04**	7.0e+02/**1.6e+02**	4.2e+05/**4.3e+03**	6.7e+02/**1.4e+02**	1.4e+07/**7.2e+05**
Total		7.5e+08/5.5e+08	1.2e+09/2.2e+09	1.3e+07/6.9e+06	1.2e+05/1.2e+05	1.3e+06/6.7e+04	5.8e+03/3.0e+04	5.6e+07/5.5e+06

Table 14: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
Find nearest Neighbour	F	13/17	24%	0.0321	**0.0250**	**1.28**×
countInRange tight_box	F	13/17	24%	0.0285	**0.0195**	**1.46**×
photonMappingBenchmark	F	12/17	29%	0.0431	0.0413	1.04×
pointCloudNeighborhood	F	11/17	35%	0.0254	**0.0164**	**1.55**×
sumMassInRange	F	14/17	18%	0.0278	**0.0201**	**1.39**×
twoPointCorrelation bin_8_16	F	11/17	35%	0.0292	**0.0206**	**1.41**×
Total				0.1860	0.1429	**1.30**×
Geomean						**1.35**×

Table 15: Per-pass PAPI counters for `KDTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
Find nearest Neighbour	F	1.4e+08/**1.1e+08**	**4.7e+08**/6.3e+08	4.6e+06/**1.9e+06**	5.5e+02/**4.9e+02**	2.2e+05/**2.4e+04**	4.1e+02/**3.8e+02**	7.4e+06/**1.5e+06**
countInRange tight_box	F	1.4e+08/**9.9e+07**	**4.0e+08**/5.5e+08	2.0e+06/**1.5e+06**	2.4e+02/**1.5e+02**	**3.0e+03**/1.8e+04	2.3e+02/**1.4e+02**	7.7e+06/**1.7e+06**
photonMappingBenchmark	F	2.2e+08/**2.1e+08**	**1.0e+09**/1.2e+09	**2.0e+06**/2.4e+06	**4.3e+02**/5.7e+02	**6.3e+03**/1.9e+04	**3.9e+02**/5.5e+02	5.6e+06/**1.5e+06**
pointCloudNeighborhood	F	1.3e+08/**8.3e+07**	**3.3e+08**/4.8e+08	2.6e+06/**7.7e+05**	2.1e+02/**1.8e+02**	**5.3e+03**/1.6e+04	1.9e+02/**1.6e+02**	7.8e+06/**1.5e+06**
sumMassInRange	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.7e+08	2.2e+06/**2.1e+06**	2.3e+02/**2.0e+02**	**4.0e+03**/2.0e+04	2.2e+02/**1.8e+02**	7.7e+06/**2.0e+06**
twoPointCorrelation bin_8_16	F	1.5e+08/**1.0e+08**	**4.8e+08**/6.1e+08	2.2e+06/**6.2e+05**	**1.5e+02**/1.8e+02	**2.9e+03**/1.3e+04	**1.3e+02**/1.8e+02	7.7e+06/**1.3e+06**
Total		9.2e+08/7.0e+08	3.1e+09/4.0e+09	1.6e+07/9.2e+06	1.8e+03/1.8e+03	2.4e+05/1.1e+05	1.6e+03/1.6e+03	4.4e+07/9.5e+06

Table 16: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
reduction	F	2/11	82%	0.0329	**0.0057**	**5.73**×
Total				0.0329	0.0057	**5.73**×
Geomean						**5.73**×

Table 17: Per-pass PAPI counters for `LinearListReduction` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
reduction	F	1.5e+08/**2.3e+07**	**9.0e+07**/1.0e+08	1.9e+06/**4.7e+04**	6.0e+02/**2.8e+02**	4.3e+05/**7.0e+03**	4.8e+02/**1.6e+02**	1.2e+07/**1.2e+06**
Total		1.5e+08/2.3e+07	9.0e+07/1.0e+08	1.9e+06/4.7e+04	6.0e+02/2.8e+02	4.3e+05/7.0e+03	4.8e+02/1.6e+02	1.2e+07/1.2e+06

Table 18: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add1 List	M	2/2	0%	**0.2567**	0.4590	0.56×
length List	F	1/2	50%	0.0414	**0.0237**	**1.75**×
sumList	F	2/2	0%	0.0440	0.0448	0.98×
sumList tail recursive	F	2/2	0%	0.0445	0.0449	0.99×
Total				0.3865	0.5723	0.68×
Geomean						0.99×

Table 19: Per-pass PAPI counters for `List` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1 List	M	**5.9e+08**/9.6e+08	**1.9e+09**/2.8e+09	8.1e+06/**2.1e+06**	**2.6e+05**/5.8e+05	**1.7e+04**/6.6e+04	**7.9e+03**/1.2e+04	9.7e+06/**7.3e+06**
length List	F	2.1e+08/**1.1e+08**	8.0e+08/**8.0e+08**	1.2e+05/**1.1e+05**	1.5e+02/**1.0e+02**	**2.7e+03**/1.4e+04	1.4e+02/**9.9e+01**	1.3e+07/**1.2e+06**
sumList	F	**2.2e+08**/2.3e+08	**9.0e+08**/1.0e+09	7.0e+05/**5.7e+05**	**1.3e+02**/1.4e+02	**3.3e+03**/6.5e+04	**1.1e+02**/1.3e+02	1.3e+07/**1.2e+07**
sumList tail recursive	F	**2.3e+08**/2.3e+08	**9.0e+08**/1.0e+09	**4.7e+05**/5.6e+05	**1.2e+02**/2.0e+02	**3.0e+03**/5.9e+04	**1.1e+02**/2.0e+02	1.3e+07/**1.2e+07**
Total		1.3e+09/1.5e+09	4.5e+09/5.6e+09	9.4e+06/3.3e+06	2.6e+05/5.8e+05	2.6e+04/2.0e+05	8.2e+03/1.2e+04	4.9e+07/3.3e+07

Table 20: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add1Tree	M	3/3	0%	0.0839	0.0766	1.10×
sumTree	F	3/3	0%	**0.0089**	0.0112	0.79×
sumTree TailRec	F	3/3	0%	**0.0084**	0.0111	0.76×
Total				0.1012	0.0989	1.02×
Geomean						0.87×

Table 21: Per-pass PAPI counters for `MonoTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1Tree	M	1.6e+08/**1.5e+08**	**5.0e+08**/5.9e+08	**3.4e+05**/5.5e+05	**3.1e+04**/4.8e+04	**4.5e+03**/7.8e+03	**1.3e+03**/2.8e+03	4.8e+05/**4.3e+05**
sumTree	F	**4.5e+07**/5.7e+07	**2.2e+08**/2.6e+08	**6.6e+03**/4.0e+04	**2.6e+02**/3.1e+02	**7.1e+02**/4.0e+03	**2.2e+02**/2.8e+02	8.8e+05/**6.8e+05**
sumTree TailRec	F	**4.3e+07**/5.6e+07	**1.9e+08**/2.2e+08	**5.8e+03**/2.5e+04	**3.2e+02**/3.4e+02	**7.7e+02**/4.7e+03	**2.8e+02**/3.2e+02	8.9e+05/**6.7e+05**
Total		2.5e+08/2.6e+08	9.1e+08/1.1e+09	3.5e+05/6.2e+05	3.2e+04/4.9e+04	5.9e+03/1.7e+04	1.8e+03/3.4e+03	2.2e+06/1.8e+06

Table 22: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
countLargeItems	F	3/5	40%	0.0435	**0.0106**	**4.09**×
countMarked	F	3/5	40%	0.0576	**0.0133**	**4.33**×
countSurvivors	F	4/5	20%	0.0135	0.0129	1.05×
deadBytes	F	4/5	20%	0.0136	0.0124	1.10×
liveBytes	F	4/5	20%	0.0203	**0.0123**	**1.65**×
sumObjIds	F	3/5	40%	0.0126	**0.0105**	**1.20**×
sweepUnmarked	M	5/5	0%	**0.0719**	0.0808	0.89×
totalHeapSize	F	3/5	40%	0.0579	**0.0190**	**3.05**×
touchHotObjects	M	5/5	0%	**0.0721**	0.0811	0.89×
Total				0.3630	0.2529	**1.44**×
Geomean						**1.66**×

Table 23: Per-pass PAPI counters for `ObjectGraph` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
countLargeItems	F	**4.7e+07**/5.3e+07	**2.6e+08**/2.8e+08	1.4e+06/**4.4e+04**	7.8e+02/**3.2e+02**	**3.3e+03**/3.7e+03	7.4e+02/**3.0e+02**	2.0e+06/**7.1e+05**
countMarked	F	**4.5e+07**/5.3e+07	**2.6e+08**/2.8e+08	1.4e+06/**4.4e+04**	4.8e+02/**3.9e+02**	**3.0e+03**/4.0e+03	4.4e+02/**3.7e+02**	1.9e+06/**6.8e+05**
countSurvivors	F	6.8e+07/**6.5e+07**	**3.1e+08**/3.8e+08	9.8e+05/**6.1e+04**	**3.0e+02**/4.4e+02	**2.4e+03**/5.6e+03	**2.6e+02**/4.0e+02	2.9e+06/**1.3e+06**
deadBytes	F	6.7e+07/**6.3e+07**	**2.7e+08**/3.5e+08	7.4e+05/**5.1e+04**	**2.9e+02**/4.4e+02	**2.4e+03**/5.5e+03	**2.7e+02**/4.1e+02	2.9e+06/**1.3e+06**
liveBytes	F	**5.6e+07**/6.3e+07	**2.7e+08**/3.5e+08	9.8e+05/**5.1e+04**	**2.8e+02**/3.7e+02	**2.5e+03**/5.5e+03	**2.4e+02**/3.5e+02	2.4e+06/**1.3e+06**
sumObjIds	F	6.4e+07/**5.3e+07**	**2.5e+08**/2.7e+08	1.1e+06/**8.5e+04**	**2.6e+02**/2.7e+02	**2.9e+03**/4.6e+03	2.4e+02/**2.4e+02**	2.9e+06/**6.9e+05**
sweepUnmarked	M	**1.9e+08**/2.4e+08	**5.5e+08**/8.8e+08	1.1e+06/**7.9e+05**	**7.3e+04**/1.0e+05	**5.7e+03**/1.3e+04	**5.6e+03**/6.5e+03	2.0e+06/**1.0e+06**
totalHeapSize	F	**4.8e+07**/5.5e+07	**2.5e+08**/2.7e+08	2.3e+06/**9.5e+05**	9.4e+02/**7.1e+02**	8.2e+04/**1.9e+04**	8.2e+02/**6.1e+02**	1.8e+06/**5.4e+05**
touchHotObjects	M	**1.9e+08**/2.3e+08	**5.7e+08**/9.0e+08	1.1e+06/**5.1e+05**	**9.7e+04**/1.2e+05	**4.7e+03**/1.1e+04	**4.6e+03**/5.8e+03	1.8e+06/**1.0e+06**
Total		7.7e+08/8.8e+08	3.0e+09/4.0e+09	1.1e+07/2.6e+06	1.7e+05/2.3e+05	1.1e+05/7.2e+04	1.3e+04/1.5e+04	2.1e+07/8.4e+06

Table 24: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
addConstPW	M	8/8	0%	0.1226	0.1345	0.91×
autorefineMaxLevel	F	4/8	50%	0.0210	0.0196	1.07×
compressMass	F	3/8	62%	0.0200	**0.0114**	**1.75**×
diffPW	M	8/8	0%	0.1206	0.1293	0.93×
lbDeuxLoadProxy	F	5/8	38%	0.0219	**0.0184**	**1.19**×
norm2Estimate	F	5/8	38%	0.0300	**0.0227**	**1.32**×
pmapCutHistogram	F	4/8	50%	0.0325	**0.0125**	**2.59**×
truncateTolViolations	F	3/8	62%	0.0203	**0.0113**	**1.80**×
Total				0.3888	0.3597	1.08×
Geomean						**1.36**×

Table 25: Per-pass PAPI counters for `PiecewiseFunctions` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
addConstPW	M	**2.7e+08**/3.3e+08	**5.7e+08**/1.2e+09	**1.4e+06**/1.7e+06	**2.6e+05**/2.6e+05	**1.6e+04**/2.6e+04	**7.4e+03**/2.4e+04	4.6e+06/**1.8e+06**
autorefineMaxLevel	F	1.1e+08/**1.0e+08**	**2.5e+08**/3.7e+08	2.5e+06/**2.1e+05**	7.5e+02/**1.4e+02**	**5.8e+03**/6.2e+03	7.5e+02/**1.2e+02**	6.2e+06/**9.3e+05**
compressMass	F	1.0e+08/**5.8e+07**	**2.5e+08**/2.8e+08	2.1e+06/**2.8e+04**	9.1e+02/**3.9e+02**	2.0e+04/**5.2e+03**	9.0e+02/**3.7e+02**	6.3e+06/**7.1e+05**
diffPW	M	**2.8e+08**/3.2e+08	**5.9e+08**/1.3e+09	**7.8e+05**/2.0e+06	**1.5e+05**/2.1e+05	**1.1e+04**/2.6e+04	**8.5e+03**/2.7e+04	4.1e+06/**1.9e+06**
lbDeuxLoadProxy	F	1.1e+08/**9.3e+07**	**3.3e+08**/4.9e+08	2.4e+06/**3.0e+05**	7.3e+02/**3.0e+02**	1.5e+04/**9.0e+03**	7.1e+02/**2.9e+02**	6.3e+06/**1.6e+06**
norm2Estimate	F	1.2e+08/**9.0e+07**	**3.4e+08**/4.4e+08	2.8e+06/**8.6e+05**	9.7e+02/**4.8e+02**	1.0e+05/**1.8e+04**	8.5e+02/**3.9e+02**	5.7e+06/**1.2e+06**
pmapCutHistogram	F	1.7e+08/**6.3e+07**	**2.8e+08**/3.7e+08	1.7e+06/**5.9e+04**	1.1e+03/**3.9e+02**	1.1e+05/**7.2e+03**	1.1e+03/**3.7e+02**	4.8e+06/**1.3e+06**
truncateTolViolations	F	1.0e+08/**5.7e+07**	**2.5e+08**/2.9e+08	2.1e+06/**3.0e+04**	7.5e+02/**4.0e+02**	1.7e+04/**4.9e+03**	7.1e+02/**3.8e+02**	6.4e+06/**7.3e+05**
Total		1.3e+09/1.1e+09	2.8e+09/4.7e+09	1.6e+07/5.2e+06	4.1e+05/4.7e+05	2.9e+05/1.0e+05	2.1e+04/5.3e+04	4.5e+07/1.0e+07

Table 26: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add 1 tree	M	5/5	0%	0.0979	0.0911	1.07×
sum tree	F	5/5	0%	0.0286	0.0270	1.06×
Total				0.1265	0.1182	1.07×
Geomean						1.07×

Table 27: Per-pass PAPI counters for `TernaryTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add 1 tree	M	2.7e+08/**2.3e+08**	**7.5e+08**/9.8e+08	**5.2e+05**/1.1e+06	**6.8e+04**/1.3e+05	**5.1e+03**/1.2e+04	**2.6e+03**/2.9e+03	1.5e+06/**1.1e+06**
sum tree	F	1.4e+08/**1.4e+08**	**4.6e+08**/5.4e+08	**7.3e+04**/2.9e+05	**1.4e+02**/1.9e+02	**8.2e+02**/9.1e+03	**1.2e+02**/1.8e+02	1.8e+06/**1.2e+06**
Total		4.2e+08/3.6e+08	1.2e+09/1.5e+09	5.9e+05/1.4e+06	6.8e+04/1.3e+05	6.0e+03/2.1e+04	2.7e+03/3.1e+03	3.2e+06/2.3e+06

Table 28: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
autocompleteTopKProxy	F	5/10	50%	0.0150	**0.0090**	**1.66**×
countLazyNodes	F	4/10	60%	0.0129	**0.0066**	**1.94**×
countTerminals	F	3/10	70%	0.0155	**0.0094**	**1.65**×
decayTrieStats	M	10/10	0%	**0.0754**	0.0867	0.87×
resetTraversalState	M	10/10	0%	0.0757	0.0810	0.93×
sumPrefixFreq	F	3/10	70%	0.0198	**0.0114**	**1.74**×
sumSubtreeHints	F	3/10	70%	0.0133	**0.0077**	**1.74**×
Total				0.2275	0.2118	1.07×
Geomean						**1.44**×

Table 29: Per-pass PAPI counters for `Trie` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
autocompleteTopKProxy	F	7.6e+07/**4.1e+07**	**1.3e+08**/2.1e+08	1.5e+06/**3.3e+05**	1.7e+02/**1.1e+02**	5.1e+04/**4.7e+03**	1.5e+02/**1.0e+02**	3.8e+06/**8.1e+05**
countLazyNodes	F	6.5e+07/**3.3e+07**	**1.3e+08**/1.6e+08	1.3e+06/**2.2e+04**	1.9e+02/**1.5e+02**	1.5e+04/**3.3e+03**	1.8e+02/**1.5e+02**	4.1e+06/**6.7e+05**
countTerminals	F	4.6e+07/**2.9e+07**	**1.0e+08**/1.2e+08	1.8e+06/**1.0e+04**	3.5e+02/**1.8e+02**	3.3e+04/**2.3e+03**	3.3e+02/**1.6e+02**	3.8e+06/**3.1e+05**
decayTrieStats	M	**1.6e+08**/2.1e+08	**3.5e+08**/7.9e+08	**7.8e+05**/1.3e+06	**9.3e+04**/1.2e+05	**5.8e+03**/2.0e+04	**2.2e+03**/2.4e+04	3.0e+06/**1.2e+06**
resetTraversalState	M	**1.6e+08**/1.9e+08	**2.8e+08**/7.0e+08	1.1e+06/**8.5e+05**	**9.2e+04**/1.0e+05	**8.2e+03**/1.4e+04	**2.9e+03**/1.7e+04	3.3e+06/**9.1e+05**
sumPrefixFreq	F	4.2e+07/**3.0e+07**	**1.1e+08**/1.2e+08	2.1e+06/**3.2e+05**	5.8e+02/**4.4e+02**	1.4e+05/**7.8e+03**	4.5e+02/**3.0e+02**	3.6e+06/**2.7e+05**
sumSubtreeHints	F	5.5e+07/**2.8e+07**	**1.1e+08**/1.2e+08	1.7e+06/**1.5e+04**	3.2e+02/**1.6e+02**	2.9e+04/**2.4e+03**	3.0e+02/**1.6e+02**	3.9e+06/**3.3e+05**
Total		6.1e+08/5.6e+08	1.2e+09/2.2e+09	1.0e+07/2.8e+06	1.9e+05/2.2e+05	2.8e+05/5.4e+04	6.5e+03/4.2e+04	2.6e+07/4.5e+06